Informalizing Lean

Aarne Ranta[1] and Claude[2]

[1] Department of Computer Science and Engineering, Chalmers University of Technology and University of Gothenburg, Sweden
aarne.ranta@cse.gu.se

[2] Claude Code, an AI coding agent by Anthropic, working in the first author's checkout

Abstract

Lean and its library Mathlib contain a large and growing part of mathematics in machine-checked form, but the formal code is hard to read for anyone who is not a Lean expert, and the code produced by autoformalization is read by even fewer. We describe a system that translates Lean declarations, with their statements, definitions and proofs, into ordinary mathematical language in five natural languages. The translation goes through the logical framework Dedukti and the Informath grammar, written in Grammatical Framework (GF), and is symbolic: it is compositional and does not invent content. The main bottleneck is the symbol table, which tells how each Lean constant is to be worded. We infer it for all of Mathlib's definitions from their docstrings and names, checking each candidate with the grammar: two thirds of 38,690 definitions get a wording. On 705 statements of Mathlib's prime numbers, orders and lattices, 83% are fully worded in English, and 265 proofs, by terms or by tactics, are rendered as texts. The system was built in a collaboration between a human researcher and an AI coding agent, which is also a coauthor of this paper; we report what came from where.

1 Introduction: the documentation problem of formal code

Formal mathematics is precise but not readable. A theorem in Lean's Mathlib [28] is stated in a language designed for an elaborator, which resolves implicit arguments, type classes, coercions and notations; a proof is a term or a script of tactics whose meaning lies in the goals that only Lean displays. Mathlib has a documentation tradition that partly compensates for this: definitions must have docstrings, and doc-gen4 [27] renders the declarations with their types. But in our checkout of Mathlib, 83% of definitions, structures and classes have a docstring, and only 15% of theorems.

The problem becomes urgent with *autoformalization*. Large language models now produce formal mathematics at a scale that no human reviewer can follow [29]. A proof checker guarantees that the code is correct, but not that it formalizes what was meant: the theorem proved may be a different theorem. To check this, someone has to read the formal statements. An LLM can be asked to explain them, but then the explanation is no more reliable than the LLM [13, 9].

Symbolic informalization is the complement: a compiler-like translation from the formal language to natural language, which follows the formal structure and adds nothing that is not there [24]. Its traditional weaknesses are fluency, the effort of building it, and the restriction to one formal language [13]. The Informath project [25] addresses them with an interlingual architecture: Dedukti [3] as the hub of formal languages and the Grammatical Framework (GF) [20, 22] for natural languages.

This paper applies Informath to Lean and Mathlib. Our contributions are:

- an exporter from Lean 4 declarations, statements, definitions and proofs to Informath's Dedukti, and a pipeline that reads them in English, French, Czech, German and Swedish, as well as in LaTeX formulas, with term proofs and tactic proofs as texts (Section 5);

- a method for inferring the symbol table, the mapping of Lean constants to words, for all of Mathlib's definitions from their docstrings and names, with each candidate checked by the grammar (Section 6);
- an account of how much comes from the existing grammar, from Mathlib's own text, from general lexical resources, from the human author and from the AI agent (Section 7);
- results on parts of Mathlib, examples good and bad, and the open problems (Sections 8–10).

The system is open source [26], with web pages showing the 705 informalized statements, a browsable table of all definitions, and a stand-alone program that informalizes any Lean file.

Authorship. The software described here and this paper were written by Claude, an AI model by Anthropic, working as a coding agent (Claude Code [2]) in the first author's checkout, in dialogue with him. The first author set the goals and directions, designed Informath before this project, made the decisions between alternatives, diagnosed some of the hardest problems, and reviewed the results; Claude wrote the code, ran the experiments and drafted the text. Section 7 makes the division more precise.

2 Lean and Mathlib: tool, resource, and test case

Lean 4 [7, 8] is a proof assistant and functional programming language based on dependent type theory. Mathlib, its community library, covers mathematics from algebra and number theory to analysis, topology and category theory; at the commit we used (`c31114ff81`, 21 October 2026) it has about 8,650 files and 2.3 million lines, with about 194,000 theorems and 48,000 definitions, structures and classes.

Mathlib plays three roles for informalization. It is a *tool*: Lean's elaborator resolves what the source leaves implicit, and its environment gives each declaration its fully elaborated type, which is what we translate. It is a *resource*: its names follow strict conventions (`Nat.Prime.two_le`, `IsPrimePow`, `minFac`), and its docstrings are written in mathematical English ("`Nat.Prime p` means that `p` is a prime number"), so that both can be mined for wordings. And it is a *test case*: it is large and varied enough that no hand-written table can cover it, and its users are an audience who can judge the results.

3 GF, Dedukti, and Informath

Informath (Figure 1) has two interlinguas. On the formal side, **Dedukti** [3] is a minimal logical framework, the $\lambda\Pi$-calculus modulo rewriting, designed as a hub between proof systems. Informath uses a fixed Dedukti theory of sets, propositions and proofs:

```
Set : Type.   Elem : Set -> Type.   Prop : Type.   Proof : Prop -> Type.
and : Prop -> Prop -> Prop.   forall : (A : Set) -> (Elem A -> Prop) -> Prop.
```

Constants beyond these base constants are user constants, such as the ones of Mathlib.

On the informal side, **GF** [20, 22] is a grammar formalism with an *abstract syntax*, a system of typed trees, and a *concrete syntax* for each language, which maps the trees to strings and back. Informath's abstract syntax has categories such as `Prop`, `Kind`, `Exp`, `Proof`, and lexical categories such as `Noun`, `Adj`, `Fun` ("the X of Y"), `Verb2`; its concrete syntaxes are built with GF's Resource Grammar Library (RGL) [21], which implements inflection, agreement and word

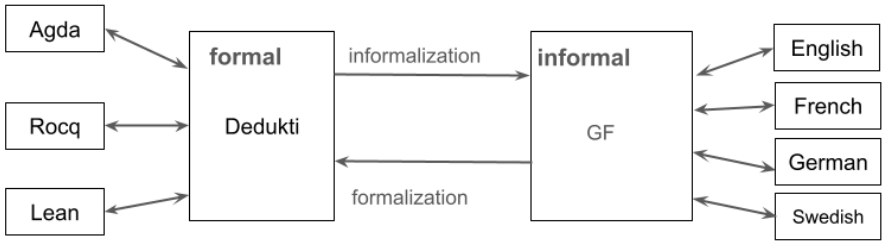


Figure 1: The architecture of Informath: Dedukti as the formal interlingua, GF as the informal one.

order for some 40 languages, so that a rule such as "x is A" is written once with `mkCl` and gets the right forms in each.

The mapping between the two is defined by a **symbol table** (`.dkgf`), which assigns to each Dedukti constant a wording, either a GF tree or an example string that the grammar parses:

```
Nat__minFac : "the smallest prime factor of #1" | $\operatorname{minFac}(#1)$
IsPrimePow  : "#1 is a prime power"
Eqt         : "#2 is equal to #3"     -- the type argument #1 is not shown
```

The string is parsed in a category `Example`, whose trees reveal the lexical category (`Fun`, `Noun1`, `Adj2`, ...) and the order of the arguments. From the table, a Dedukti statement is translated to a GF tree in a core language (MathCore), then rewritten by natural language generation (NLG) rules into variants (in situ quantification, aggregation of hypotheses, symbolic notation where it fits), which are ranked and linearized [24]. A constant without an entry is shown symbolically, as IsUnit(u).

4 Prior work

Readable formal mathematics has a long history. Mizar [4] was designed to look like mathematical text and produces the journal *Formalized Mathematics*. Coscoy et al. [5] extracted texts from Coq (now Rocq) proof terms, which is close to what we do for Lean's term proofs. Isabelle/Isar [30] is a readable proof language; Verbose Lean [16] gives Lean a controlled natural language for teaching, in which tactic proofs read like texts, and inspired our treatment of tactic proofs. In the other direction, controlled natural languages such as Naproche [6, 15], ForTheL [18] and MathNat [12] are parsed into formal languages. GF has been used for mathematical language from the start [10, 23, 14], and GFLean [19] autoformalizes a fragment of English into Lean with GF.

Neural approaches have used LLMs to informalize Lean, notably the Herald dataset of Mathlib statements annotated in natural language [9], and to autoformalize, with symbolically generated data as one source [13, 11]. Our work is complementary: the translation itself is symbolic, but the lexicon that it needs is inferred from Mathlib's own text, and the system was built with an AI agent.

5 From Lean code to natural language

The pipeline has four steps.

1. Export to Dedukti. A Lean program, `Export.lean`, imports Mathlib modules and translates each declaration's elaborated type, and the definiens of definitions and term proofs, to Informath's Dedukti. A binder $(x : \alpha)$ with α : `Type` becomes $(x$: `Elem` $\alpha)$, one with α : `Prop` a hypothesis, and $(\alpha$: `Type` $u)$ becomes $(\alpha$: `Set`$)$. Implicit arguments are dropped in applications, so that only the arguments a reader sees in Mathlib remain. An instance binder `[CommMonoid M]` becomes a hypothesis `Proof (CommMonoid M)`, a class is declared as a predicate, and a bundled function applied by coercion (`DFunLike.coe f x`) becomes a constant `f__app` with its own type. A table of 39 mappings sends Lean's logic and arithmetic (`Dvd.dvd`, `Nat.Prime`, `HAdd.hAdd`, `Iff`) to Informath's base constants, whose wordings are part of Informath. The output is meant to be read, not checked: dropping implicit arguments loses information a type checker would need. For example, the Lean theorem

```
theorem Irreducible.not_dvd_isUnit {M : Type*} [CommMonoid M] {p u : M}
    (hp : Irreducible p) (hu : IsUnit u) : ¬p ∣ u
```

is exported with its implicit variables and its instance as hypotheses, as a theorem is stated with all of them, and its proof term with them as λs:

```
def Irreducible__not_dvd_isUnit : (M : Set) -> (_ : Proof (CommMonoid M)) ->
    (p : Elem M) -> (u : Elem M) -> (hp : Proof (Irreducible p)) ->
    (hu : Proof (IsUnit u)) -> Proof (not (divide p u)) :=
  M => x => p => u => hp => hu =>
    mt (divide p u) (IsUnit p)
       (ifI (divide p u) (IsUnit p) (h => isUnit_of_dvd_unit M p u h hu))
       (Irreducible__not_isUnit M p hp).
```

Here `divide` and `not` are Informath's base constants, mapped from `Dvd.dvd` and `Not`, and `ifI` is the natural deduction rule of implication introduction. The formula reading of the statement is $M \in \mathrm{Set}, \mathrm{CommMonoid}(M), p, u \in M, \mathrm{Irreducible}(p), \mathrm{IsUnit}(u) \vdash \neg(p \mid u)$; its verbal reading is shown in Section 8.

2. Readings. RunInformath translates the Dedukti declarations with the base table, Mathlib's table (Section 6) and the grammar, and gives a reading in each language and a formula in LaTeX. The index of all 705 statements is built in one call per mode, so that the grammar of five languages (8 MB) is loaded once. German linearization turned out to be exponential in nested noun phrases (three levels of "the maximum of" took 35 seconds), so German is read apart, with a time limit of one second per statement.

3. Term proofs. A definiens that is a proof term is translated to a sequence of proof units: each λ becomes an assumption ("Let n be an element of $\mathbb{N}$", "Assume that p is prime"), each application of a lemma a conclusion, justified by the lemma. Lean's logical constructors are mapped to natural deduction rules in Dedukti.

4. Tactic proofs. Most of Mathlib's proofs are tactic scripts, whose text says little: `rw [hn, mul_one]`. Following the idea of Verbose Lean [16], we use the goals instead. The exporter elaborates the module again with Lean's info trees, and records for each tactic of a sequence the goal before it and the goals after it, translated to Dedukti. A step then becomes units by comparing them: new hypotheses are assumed or stated, a closed goal is a conclusion, a changed goal is "it is enough to show that ...", several goals are cases. Each unit keeps the tactic as its justification:

```
theorem Prime.eq_one_or_self_of_dvd {p : ℕ} (pp : p.Prime) (m : ℕ) (hm : m ∣ p) :
    m = 1 ∨ m = p := by
  obtain ⟨n, hn⟩ := hm
  have := pp.isUnit_or_isUnit hn
  rw [Nat.isUnit_iff, Nat.isUnit_iff] at this
  apply Or.imp_right _ this
  rintro rfl
  rw [hn, mul_one]
```

1.	Let n be an element of $\mathbb{N}$ such that $p = m \times n$.	`obtain ⟨n, hn⟩ := hm`
2.	m is a unit or n is a unit.	`have := pp.isUnit_or_isUnit hn`
3.	$m = 1$ or $n = 1$.	`rw [Nat.isUnit_iff, Nat.isUnit_iff] at this`
4.	It is enough to show that if $n = 1$, then $m = p$.	`apply Or.imp_right _ this`
5.	Assume that $n = 1$.	`rintro rfl`
6.	$m = p$.	`rw [hn, mul_one]`

The same units are read in the other languages: "Stačí ukázat, že jestliže $n = 1$, pak $m = p$", "Es genügt zu zeigen, dass ...".

Outputs. The results are shown on web pages, one per part of Mathlib, where each declaration has its Lean source, its Dedukti translation (folded), its readings and formula, and its proof text [26]. A stand-alone program, `mathlib-informath`, does the same for any Lean file or code on standard input, as plain text, HTML or PDF, in about 40 seconds, most of which Lean spends loading Mathlib.

For instance, the code on standard input

```
theorem t (n : ℕ) (h : 0 < n) : n ≠ 0 := by omega
```

gives, with `-languages=Eng,Ger`,

```
t (theorem)
  English: Let $n$ be a natural number such that $0 < n$. Then $n \neq 0$.
  German: Sei $n$ eine natürliche Zahl derart, dass $0 < n$. Dann $n \neq 0$.
  Proof, English:
    1. $n \neq 0$.   [omega]
```

A user's own symbol table can be added on top of Mathlib's, for the concepts that the code defines itself.

6 Inferring the symbol tables

Informath's own lexicon covers basic mathematics; Mathlib has tens of thousands of concepts. The first tables were written by hand: 108 entries for the constants that our first modules use, with suggestions from docstrings and names checked by the grammar. This does not scale, so we infer a table for all of Mathlib's definitions.

The definitions. `export -definitions` imports all of Mathlib (20 seconds) and exports every declaration that may need a wording: definitions, structures, classes, fields, inductive types, but not theorems, instances, tactics, notations, auxiliary or deprecated declarations. Of 56,867 such constants, 38,690 remain after filtering, each with its Dedukti type, explicit argument names and docstring. Its type tells its category: a function or constant (`Exp`), a type (`Kind`), or a predicate (`Prop`).

Candidates. For each definition, candidate wordings are made in two ways. *From the docstring*, patterns pick noun phrases and predicates: "`‘f n‘` is the X of `‘n‘`", "The X of ... is", "A set is X if", with the backticked argument names replaced by their positions `#1`, `#2`. *From the name*, the last component is split at capitals and underscores, and Mathlib's abbreviations are spelled out (`minFac`: minimum factor; `card`: cardinality; `AddMonoidHom`: additive monoid homomorphism, with *Add* as a modifier rather than "sum"). The name gives "the X of `#1`" for a function, "X of `#1`" for a type, "`#1` is (a) X" for a predicate (`IsCountablyCompact`: "`#1` is countably compact"), "`#1` has X" for one named after a verb (`HasLimits`, `PreservesFiniteLimits`). A few conventions needed special rules: a constructor `Bicone.ofLimitCone` is named after its namespace ("the bicone of `#1`"), a function `pointOfClosedPoint` after its head, but a type `PartitionOfUnity` keeps its "of".

For example, `Ordinal.cof : Ordinal → Cardinal` has the docstring "The cofinality on an ordinal is the `Order.cof` of any isomorphic linear order". Its first pattern gives "the cofinality on an ordinal of `#1`", which does not parse; with the generic prepositional phrase cut out, "the cofinality of `#1`" does, and it is also what the name gives, as `cof` is spelled out as *cofinality*. Since the candidate shares a word with the name, it is accepted; a docstring wording that shares no word with the name, such as "the ring" for `HahnSeries.toPowerSeries`, is not tried.

Checking. All 39,000 candidates are parsed by the grammar in one call (30 seconds). A candidate is accepted if its parse has the category and the arity of the constant; the first one accepted becomes the entry. Many constants take explicit parameters before their main arguments (a category, a Grothendieck topology); a wording may leave them out, as `Eqt` above leaves out the type. But leaving out the wrong argument gives a false reading: `IsRelUpperSet s P` was first read "P is a relation upper set". A predicate of two objects is therefore read as binary, "`#1` is a relative upper set with respect to `#2`", with the subject chosen by the docstring (the argument that it does *not* name after "relative to", "for", "over"), and without such a clue it gets no reading. We prefer a symbolic reading to a wrong one.

The lexicon. Candidates fail mostly because of unknown words. These are added to the grammar: a word that recurs in the candidates gets a noun, adjective, adverb or verb entry, with its category and plural from the RGL's English dictionary (`DictEng`), and a word that the dictionary lacks is admitted if three docstrings use it. Junk had to be filtered out by hand-made lists: docstrings contain function words that the dictionary lacks (*because*, *without*) and Lean jargon (*simproc*, *defeq*). The result is 507 English words, from *cocone*, *equicontinuous* and *hypercover* to *Zariski*.

For the other languages, we first tried GF's WordNet [17, 1], choosing senses by their domain tags: only 117 of 789 words were found, 13 of them in a sense tagged *mathematics*, and even those were mixed (*hexagon*: *Sechseck*, correct; *minus*: *Weg*, wrong; *triplet*: *trois*, wrong). Mathlib's jargon is not there. We therefore keep the translations in a table, one line per word with its French, German, Swedish and Czech forms (gender, plural), written by Claude and to be reviewed, with an empty cell where unsure; the grammar modules are generated from it. Where a language lacks a word, the index keeps the reading made without the generated table, which shows the constant symbolically.

7 Who contributed what

Table 1 shows where the wordings of the definitions come from: most from names, an eighth from docstrings, and a few from the hand-written table, which overrides the generated one. Table 2

	functions, constants	types	predicates	all
definitions	31,877	2,466	4,347	38,690
with a wording	22,276	1,763	1,818	**25,857** (67%)
the wordings from: names 22,307; docstrings 3,488; the hand-written table 62				

Table 1: The symbol table of Mathlib's definitions.

Already in Informath	**From the first author**	**By Claude**
The GF grammar and its concrete syntaxes on the RGL; the Dedukti theory; symbol tables with verbal, symbolic and macro forms; NLG with variants and ranking; proof texts from terms	The goal; the choice of Mathlib's prime numbers as the first case; the contents of the pages; loading the grammar once; the idea of tactic proofs as proof units; the time limit for German; WordNet as a resource; a reviewable table rather than generated code; the stand-alone program	`Export.lean` with its translation rules; the index and the pages; the table inference and the lexicon generation; the translations table; the Informath extensions this needed (multilingual proof texts, `-tactic-proof`, aggregated hypotheses, Czech and Swedish modules)

Table 2: The division of labour.

divides the work between the existing Informath, the first author's prompts and decisions, and the AI agent.

Some of the decisive steps were the human's: the diagnosis of a parameter explosion in the GF compiler (a list category whose concrete functions grew to 5×10^{13}), the reason why size-optimized grammars broke all proof texts (the proof units were not reachable from the start category), and the decision to keep the translations reviewable. Others were the agent's: the iteration over hundreds of failing cases, measuring each change on the whole of Mathlib, and testing a resource (WordNet) before relying on it. All the words written by Claude, in all languages, are to be reviewed by native speakers.

8 Examples

Definitions. Mathlib defines a prime number as an irreducible element:

```
def Prime (p : ℕ) := Irreducible p
```

> Let p be a natural number. Then p is prime, if p is irreducible.
> Soit p un nombre naturel. Alors p est premier si p est irréductible.
> Låt p vara ett naturligt tal. Då är p primt, om p är irreducibelt.

Wordings from the generated table, with the arguments shown by their names: "the cofinality of o", "the normalizer of H", "C has limits", "F preserves finite limits", "P is separated with respect to J", "M is a perfect matching".

Theorems. Euclid's theorem in five languages:

```
theorem exists_infinite_primes (n : ℕ) : ∃ p, n ≤ p ∧ Prime p
```

> Let n be a natural number. Then there exists a natural number p, such that $n \leq p$ and p is prime.
> Soit n un nombre naturel. Alors il existe un nombre naturel p tel que $n \leq p$ et p soit premier.
> Sei n eine natürliche Zahl. Dann gibt es eine natürliche Zahl p, derart, dass $n \leq p$ und p prim ist.
> Låt n vara ett naturligt tal. Då finns det ett naturligt tal p, så att $n \leq p$ och p är primt.
> Nechť n je přirozené číslo. Tedy existuje přirozené číslo p, takový, že $n \leq p$ a p je prvočíselné.

The aggregation of hypotheses makes statements with many variables readable:

> Let M be a set such that M is a commutative monoid. Let p and u be elements of M such that p is irreducible and u is a unit. Then p does not divide u.

(`Irreducible.not_dvd_isUnit`; the Lean statement has the implicit `M`, `p`, `u` and the instance `[CommMonoid M]`.)

Term proofs. The definiens of `exists_infinite_primes`, with its local definitions and `have`s, becomes a proof text of twelve lines, which starts:

> 1. Let n be an element of $\mathbb{N}$. 2. $0 < n!$. 3. The successor of 0 is less than the successor of $n!$. 4. $n! + 1 \neq 1$. 5. minFac$(n! + 1)$ is prime. ...

Each line is justified by the lemma it applies (`Nat.factorial_pos`, `Nat.succ_lt_succ`, ...), linked to its own entry.

Bad cases. Not everything reads well:

- a constant without a wording stays symbolic: "Let n be a natural number such that Coprime$(2, n)$" in French, where the English has "2 and n are coprime";
- mismatched agreement: Czech *takový* above, which should be *takové*; "Let s be a upper set";
- literal wordings: Swedish "lika till" for "equal to" (*lika med*), as the table's "is equal to" is parsed word by word;
- generated wordings that say little: `BoundedOrderHom.dual`, a bundled map without explicit arguments, is "the dual";
- German: "Dann $2 \leq p$" without a verb, and 51 statements over the time limit.

9 Results

Table 3 shows the readings of 705 statements from 19 modules: Mathlib's order and lattice definitions, the algebra that the prime numbers import, the prime numbers, and prime factorizations. A statement is *fully worded* if no Lean constant in it is shown symbolically. Every statement has a formula, and 264 declarations come with their definiens. Of the 242 term proofs, 240 have a proof text in each language; of the 242 tactic proofs, the 25 of `Nat.Prime.Defs` have one so far, in five languages. In 26 statements a language keeps the older, more symbolic reading because it lacks a generated word. The whole index takes about ten minutes on a laptop, most of it for German.

	statements	English	French	Czech	German	Swedish
Order, lattices	354	291	289	289	248	289
Algebra	29	29	29	29	28	28
Prime numbers	128	125	125	125	125	125
Factorizations	194	143	143	143	132	142
all	705	**588**	**586**	**586**	**533**	**584**

Table 3: Fully worded statements, by language.

	examples	
good	"C has injective resolutions", "f is differentiable", "the wide pullback of s", "the Grothendieck topology of T", "the smallest prime factor of x"	22
acceptable	"the evaluation of g and p" (`MvPolynomial.eval`$_2$), "the adjunction", "the tail of p and *len_pos*" (a proof argument shown)	20
wrong	"the vector multiplicative vector of w and v" (`vecMulVec`), "the unit product of u and w" (`unitsSMul`), "the opposite opposite of S" (`opUnop`), "R is a continuous square root" (a class saying that one exists)	18

Table 4: A random sample of 60 generated wordings, judged by the second author.

For all of Mathlib, the generated table gives two thirds of the definitions a wording (Table 1), checked by the grammar for category and arity but not reviewed by hand. Random samples during the development showed classes of errors, which led to the rules of Section 6.

A sample. To estimate the quality of the final table, Claude judged a random sample of 60 generated wordings against the definitions' docstrings (Table 4): a wording is *good* if it says what the definition is in idiomatic terms, *acceptable* if it is vague but not wrong, and *wrong* if it says something else or nothing. This is the judgement of one annotator, the second author, and is not yet reviewed.

Two thirds are thus usable, and the predicates fare best: of the eleven in the sample, nine are good. The wrong ones are mostly composite names read word by word (`presheafHomSectionsEquiv`: "the presheaf homomorphism section equivalence"), and abbreviations spelled out wrongly in context (*SMul* is a scalar multiplication, not a product). Proof arguments and unnamed arguments (#4) should not be shown, and `LibraryNote`, documentation without mathematical content, should have been filtered out.

10 Known problems and open issues

Coverage. 705 statements are a small part of Mathlib's 240,000 declarations, and the generated table is unreviewed. Composite names (`quotientIdealMapEquivTensorQuot`) become nonsense when read word by word; bundled maps without arguments get bare readings; about 2,500 unknown words are abbreviations of jargon that we do not spell out. Most tactic proofs have no text yet, and long `simp` goals make poor reading.

Language. Agreement of "such that" with an existential's kind fails in Czech and French; multi-letter variables (`self`, `motive`) are shown as constants; a lemma used unapplied in a proof term gives garbled text; the meaning of $\leq$ on finite sets is "subset", which the reading does not know. The words of four languages need native review, and German linearization needs a fix in the RGL.

Correctness. The translation is compositional, but the exporter drops implicit arguments, so the Dedukti code is not type-checked; a reading is as correct as the table entries it uses, and the generated entries are checked only by the grammar. A back-translation from the readings to Dedukti and a type check would close this loop, as Informath supports in the other direction.

11 Conclusion

Symbolic informalization of Lean is feasible at the scale of Mathlib if the lexicon can be inferred: Mathlib's naming conventions and docstrings give wordings to two thirds of its definitions, and a grammar can check them. The readings are faithful by construction, and they come in five languages at once. What the method does not give is judgement: which wording is right, which word a language uses, when a reading is false although it parses. That remained the human's part, in a collaboration where the AI agent did most of the construction and the measuring.

Acknowledgements. The first author thanks Krasimir Angelov for the RGL work on Czech and German that this project relied on, and Josef Urban for introducing him to agentive coding. This work has been supported by the European Research Council through the grants *NextReason* (Grant ID 101200949) and *MALINCA* (Grant ID 101167526).

References

[1] Krasimir Angelov. A parallel WordNet for English, Swedish and Bulgarian. In *Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020)*, pages 3008–3015, 2020.

[2] Anthropic. Claude Code. https://claude.com/claude-code, 2025.

[3] Ali Assaf, Guillaume Burel, Raphaël Cauderlier, David Delahaye, Gilles Dowek, Catherine Dubois, Frédéric Gilbert, Pierre Halmagrand, Olivier Hermant, and Ronan Saillard. Dedukti: a logical framework based on the $\lambda\pi$-calculus modulo theory, 2023.

[4] Grzegorz Bancerek, Czesław Byliński, Adam Grabowski, Artur Korniłowicz, Roman Matuszewski, Adam Naumowicz, Karol Pak, and Josef Urban. Mizar: State-of-the-art and beyond. In Manfred Kerber, Jacques Carette, Cezary Kaliszyk, Florian Rabe, and Volker Sorge, editors, *Intelligent Computer Mathematics*, pages 261–279, Cham, 2015. Springer International Publishing.

[5] Yann Coscoy, Gilles Kahn, and Laurent Thery. Extracting text from proofs. In Mariangiola Dezani-Ciancaglini and Gordon Plotkin, editors, *Proc. Second Int. Conf. on Typed Lambda Calculi and Applications*, volume 902 of *LNCS*, pages 109–123, 1995.

[6] Marcos Cramer, Bernhard Fisseni, Peter Koepke, Daniel Kühlwein, Bernhard Schröder, and Jip Veldman. The Naproche Project Controlled Natural Language Proof Checking of Mathematical Texts. In Norbert E. Fuchs, editor, *CNL*, volume 5972 of *LNCS*, pages 170–186. Springer, 2009.

[7] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In Amy P. Felty and Aart Middeldorp, editors, *Automated Deduction - CADE-25*, pages 378–388, Cham, 2015. Springer International Publishing.

[8] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.

[9] Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. Herald: A natural language annotated Lean 4 dataset. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[10] Thomas Hallgren and Aarne Ranta. An Extensible Proof Text Editor. In Michel Parigot and Andrei Voronkov, editors, *LPAR-2000*, volume 1955 of *LNCS/LNAI*, pages 70–84. Springer, 2000. http://www.cse.chalmers.se/~aarne/articles/lpar2000.pdf.

[11] Pei Huang, Nicholas Smallbone, and Aarne Ranta. Multilingual Autoformalization via Fine-tuning Large Language Models with Symbolically Generated Data. *RISC Proceedings on Symbolic Computation and Machine Learning*, 1:3–20, 2025.

[12] Muhammad Humayoun and Christophe Raffalli. MathNat - Mathematical Text in a Controlled Natural Language. *Journal on Research in Computing Science*, 66, 2010.

[13] Albert Q. Jiang, Wenda Li, and Mateja Jamnik. Multi-language diversity benefits autoformalization. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 83600–83626. Curran Associates, Inc., 2024.

[14] Josefin Kelber, Michael Kohlhase, Jan Frederik Schaefer, and Marcel Schütz. Reaping the benefits of modularization in flexiformal mathematics by GF-based AST transformations. In *Intelligent Computer Mathematics: 18th International Conference, CICM 2025, Brasilia, Brazil, October 6–10, 2025, Proceedings*, page 191–207, Berlin, Heidelberg, 2025. Springer-Verlag.

[15] Peter Koepke. A Natural Language Formalization of Perfectoid Rings in Naproche. In Yannick Forster and Chantal Keller, editors, *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 6:1–6:15, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[16] Patrick Massot. Teaching Mathematics Using Lean and Controlled Natural Language. In Yves Bertot, Temur Kutsia, and Michael Norrish, editors, *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:19, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[17] George A. Miller. WordNet: A Lexical Database for English. *Commun. ACM*, 38(11), 1995.

[18] Andrei Paskevich. The Syntax and Semantics of The ForTheL Language, 2007.

[19] Shashank Pathak. GFLean: An Autoformalisation Framework for Lean via GF. arXiv 2404.01234, 2024.

[20] Aarne Ranta. Grammatical Framework: A Type-Theoretical Grammar Formalism. *The Journal of Functional Programming*, 14(2):145–189, 2004.

[21] Aarne Ranta. The GF Resource Grammar Library. *Linguistics in Language Technology*, 2, 2009.

[22] Aarne Ranta. *Grammatical Framework: Programming with Multilingual Grammars*. CSLI Publications, Stanford, 2011.

[23] Aarne Ranta. Translating between language and logic: What is easy and what is difficult. In Nikolaj Bjørner and Viorica Sofronie-Stokkermans, editors, *Automated Deduction – CADE-23*, pages 5–25, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[24] Aarne Ranta. Towards multilingual autoformalization and informalization of mathematics. In *SLTC-2024, Swedish Language Technology Conference*, 2024. https://sltc2024.github.io/abstracts/ranta.pdf.

[25] Aarne Ranta. Informath: Informalization and Autoformalization of Formal Mathematics. https://github.com/GrammaticalFramework/informath, 2026.

[26] Aarne Ranta and Claude. mathlib-informath: Informalizations of Lean Mathlib4. https://github.com/aarneranta/mathlib-informath, 2026.

[27] The Lean community. doc-gen4: Document generator for Lean 4. https://github.com/leanprover/doc-gen4, 2025.

[28] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.

[29] Josef Urban. 130k lines of formal topology in two weeks: Simple and cheap autoformalization for everyone? arXiv 2601.03298, 2026.

[30] Markus Wenzel. Isar - a generic interpretative approach to readable formal proof documents. In Yves Bertot, Gilles Dowek, André Hirschowitz, C. Paulin, and Laurent Théry, editors, *TPHOLs*, volume 1690 of *LNCS*, pages 167–184. Springer, 1999.